

KUBERNETES TECHNICAL REPORT

Generated using LangGraph Multi-Agent AI

Generated: 01 October 2026, 03:49 PM

User Question

what is kubernetes? and make workflow for ai engineer

Final Report

Technical Report

****Kubernetes for AI Engineering – Architecture, Workflow, and Best Practices****

Prepared for: AI Engineering Team

Date: 1 Oct 2026

1. Executive Summary

Kubernetes (k8s) is the de facto platform for running containerised workloads at scale. For AI engineers it delivers reproducible model training, elastic inference, and robust lifecycle management while abstracting the underlying infrastructure. This report synthesises the core concepts of Kubernetes, maps them to AI specific use cases, and presents a concrete workflow that an AI engineer can adopt from code to production. It also outlines the technical architecture, deployment strategy, security posture, monitoring stack, implementation steps, benefits, challenges, and a concise conclusion.

2. Research Findings

2.1 What is Kubernetes?

| Concept | What it is | Why it matters |

|-----|-----|-----|

| ****Cluster**** | A set of machines (physical or virtual) that run Kubernetes. | Provides the pool of resources (CPU, memory, storage) that workloads consume. |

| **Node** | A single machine in a cluster. Each node runs a **kubelet** agent and container runtime (e.g., Docker, containerd). | Executes the containers that make up applications. |

| **Pod** | The smallest deployable unit; one or more containers that share the same network namespace and storage volumes. | Encapsulates an application component and its dependencies. |

| **Deployment** | Declarative object that manages a set of replica Pods. | Handles rolling updates, rollbacks, and scaling. |

| **Service** | Abstraction that defines a logical set of Pods and a policy to access them (e.g., ClusterIP, NodePort, LoadBalancer). | Provides stable networking and load balancing. |

| **Ingress** | Collection of rules that allow inbound connections to reach Services. | Manages HTTP/HTTPS routing, TLS termination, and virtual hosting. |

| **ConfigMap / Secret** | Key-value stores for configuration data and sensitive information. | Keeps code separate from environment-specific settings. |

| **Namespace** | Virtual partition of cluster resources. | Enables multi-tenant or multi-environment isolation. |

| **Horizontal Pod Autoscaler (HPA)** | Automatically adjusts the number of Pods based on metrics (CPU, memory, custom). | Keeps workloads responsive while optimising resource usage. |

| **PersistentVolume / PersistentVolumeClaim** | Abstraction for storage that can outlive Pods. | Provides durable data for stateful workloads. |

2.2 Requirements for Using Kubernetes

Requirement	Detail
Infrastructure	Physical servers, cloud VMs, or managed services (GKE, EKS, AKS, DigitalOcean Kubernetes, etc.).
Container Runtime	Docker (deprecated in newer k8s), containerd, or CRI-O.
Cluster Management	<code>kubeadm</code> , managed services, or tools like <code>kops</code> , <code>k3s</code> , <code>microk8s</code> .
Networking	CNI plugin (Calico, Flannel, Weave, Cilium) to provide pod networking.
Storage	CSI drivers for block/filestore (e.g., EBS, GCE PD, Azure Disk).
Security	RBAC, NetworkPolicies, PodSecurityPolicies/OPA, Secrets management.
Monitoring & Logging	Prometheus, Grafana, ELK/EFK stack, or managed observability services.
CI/CD	GitOps tools (ArgoCD, Flux), Helm, or traditional pipelines (Jenkins, GitHub Actions).
Domain Knowledge	Understanding of containers, YAML manifests, and Kubernetes primitives.

2.3 Benefits of Kubernetes for AI Engineering

Benefit	Explanation
---------	-------------

|-----|-----|

- | **Scalable Deployment** | Auto-scale inference services based on traffic or GPU utilisation. |
- | **Resource Isolation** | Separate CPU/GPU quotas per Pod; prevents “noisy neighbour” effects. |
- | **Reproducibility** | Declarative manifests and container images ensure identical environments across dev, test, prod. |
- | **CI/CD Integration** | Automated image builds, Helm chart releases, and GitOps pipelines reduce manual errors. |
- | **Multi-Cloud / Hybrid Flexibility** | Same deployment artifacts run on any Kubernetes-compatible platform. |
- | **Observability** | Built-in metrics, logs, and tracing hooks for monitoring model performance and latency. |
- | **Model Lifecycle Management** | Versioned deployments, canary releases, and rollback capabilities. |
- | **Cost Optimization** | Spot instances, node autoscaling, and efficient GPU sharing reduce infrastructure spend. |

2.4 Challenges / Pain Points

| Challenge | Mitigation |

|-----|-----|

- | **Complexity of Setup** | Use managed Kubernetes services or lightweight distributions (k3s, microk8s). |
- | **GPU Scheduling** | Requires specialised device plugins (NVIDIA Device Plugin) and careful resource requests/limits. |
- | **Stateful Workloads** | Persistent volumes and StatefulSets add operational overhead. |
- | **Security** | Need to harden cluster, manage secrets, and enforce network policies. |
- | **Observability Overhead** | Collecting metrics from many Pods can be expensive; use sampling or selective instrumentation. |
- | **Learning Curve** | Requires knowledge of YAML, kubectl, and cluster concepts; invest in training. |
- | **Version Compatibility** | Keep cluster and tooling (kubectl, Helm) in sync; use versioned APIs. |

2.5 Use Cases for AI Engineers

| Use Case | How Kubernetes Helps |

|-----|-----|

- | **Batch Model Training** | Use Jobs or CronJobs to run training jobs on GPU nodes; automatically clean up after completion. |
- | **Online Inference** | Deploy inference containers behind a Service; autoscale based on latency or request rate. |

| **Model Serving with Multiple Versions** | Use separate Deployments/Services for A/B testing or canary releases. |

| **Feature Store** | Run a distributed feature store (e.g., Feast) as a StatefulSet with persistent storage. |

| **Experiment Tracking** | Run MLflow or Weights & Biases in a pod; expose UI via Ingress. |

| **Data Pipeline Orchestration** | Combine Airflow or Prefect with KubernetesExecutor to run tasks in isolated Pods. |

| **Edge Deployment** | Use K3s or MicroK8s on edge devices to run lightweight inference workloads. |

2.6 Sample Workflow for an AI Engineer Using Kubernetes

1. **Local Development** – Write model code, test locally with Docker.
2. **Containerisation** – Build a Docker image, push to a registry.
3. **Infrastructure Preparation** – Configure cluster access, namespaces, RBAC, GPU support.
4. **Deployment Configuration** – Create a Helm chart or Kustomize overlay.
5. **CI/CD Pipeline** – Build image → push → Helm upgrade via ArgoCD/Flux.
6. **Runtime Operations** – Expose metrics, logs, tracing; enable autoscaling and canary releases.

3. Technical Architecture

...



